北京大学
PEKING UNIVERSITY

Review of *Lec21 Dynamic Memory Allocation*

吴悠[1]

[1]School of Physics, Peking University

✉ 2300011348@stu.pku.edu.cn

Dec 10, 2025

北京大学 PEKING UNIVERSITY

Contents

1. Basic concepts
2. Implicit free list(隐式空闲链表)
3. Explicit free list(显式空闲链表)
4. Segregated free list(分离空闲链表)
5. 程序设计中和memory有关的坑
6. Garbage collection

北京大学
PEKING UNIVERSITY

PART 01

Basic concepts

- *Explicit allocator & Implicit allocator*
- *Throughput & peak memory utilization*
- *Block, payload & fragmentation*
- *Block size, header & footer*

Block & alignment

1. Malloc (不初始化它返回的内存)
2. Calloc (初始化为0)
3. Realloc
4. Pointers returned by malloc are double-word aligned
5. 一个word是多少？书上(以32位为例)是4byte，ppt上8byte
6. 总之这里word ＝machine word
7. sbrk：扩展和收缩堆，返回brk的旧值 （失败返回-1，设置errno）
8. 一个正方形一个字，一个块必须偶数个正方形

Explicit allocator

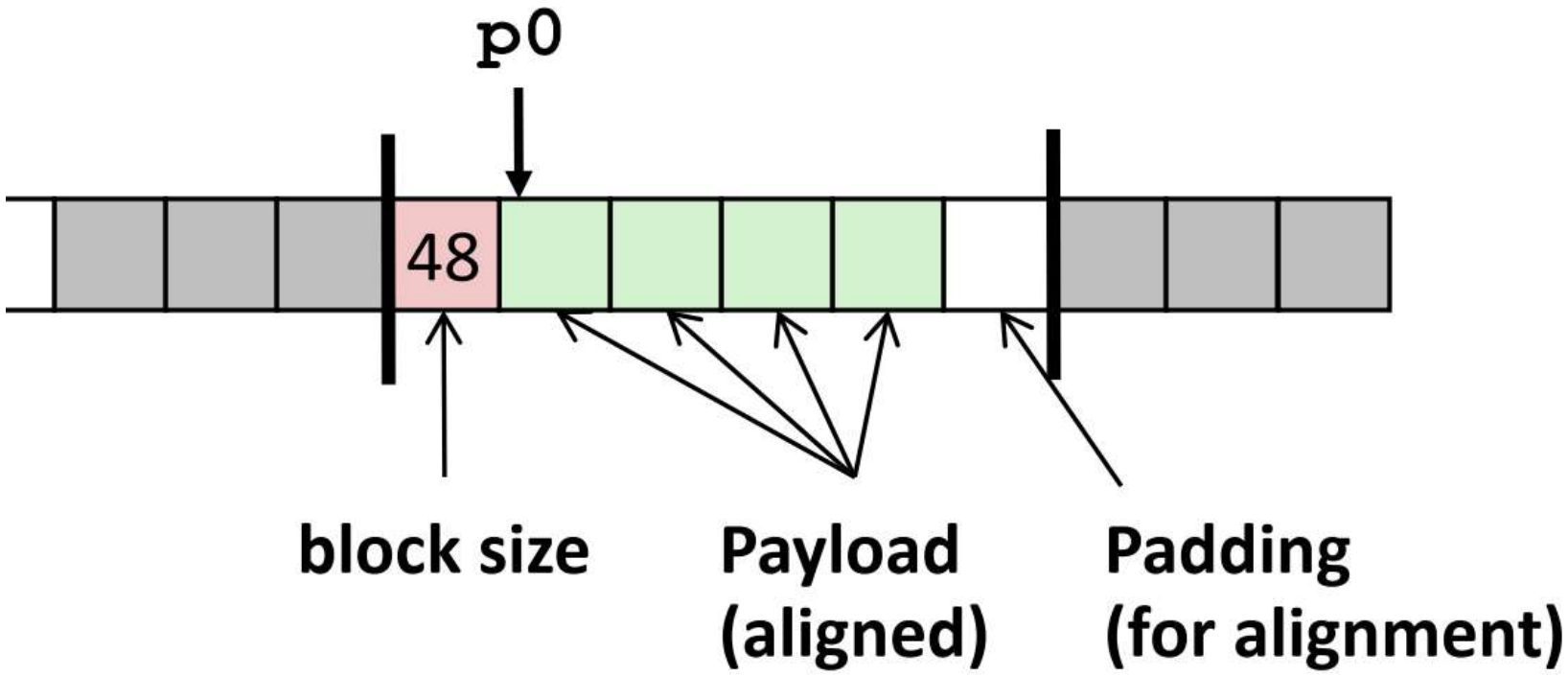
显式分配器: malloc, new
Malloc要求调用free来释放，new要求delete释放
隐式分配器（垃圾收集器，自动释放未使用的已分配的块的过程叫做垃圾收集）

显式还是隐式，关键看要不要显式释放

对explicit allocator的约束

1. 处理任意请求序列（分配和释放，只能释放已分配的块）
2. 立即响应请求（不可以重排、缓冲）
3. 只使用堆
4. 对齐块(对齐要求)
5. 不修改已分配的块（什么叫不修改？移动、压缩都不可以）

What a block is like



头部+有效载荷+填充

Performance goal

1. Maximize throughput

Throughput的定义：Number of completed requests per unit time

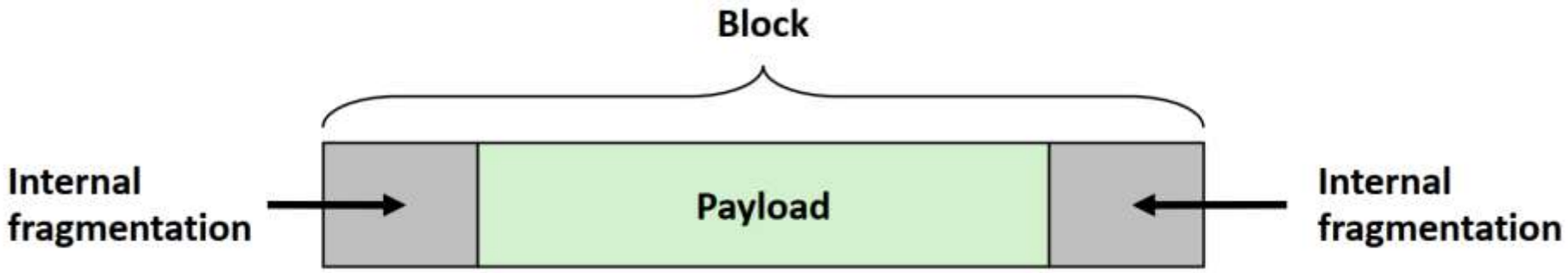
2. Maximize peak memory utilization(峰值利用率)
3. Payload (有效载荷)

Then the peak utilization over the first $k+1$ requests, denoted by U_k, is given by

$$U_k = \frac{\max_{i \le k} P_i}{H_k}$$

Fragmentation (碎片)

1. 内部碎片：头部、对齐填充
2. 外部碎片：从整个内存的占用和空闲来看，够放，但没有一个单独的块够放；不好量化，取决于你放什么



怎么分配块?

1. First fit:从头开始搜索空闲链表，选择第一个合适的空闲块
2. Next fit:从上一次查询结束的地方开始搜索空闲链表，选择第一个合适的空闲块
3. Best fit:检查每个空闲块，选择适合所需请求大小的最小空闲块
4. 怎么查这些free blocks?

空闲链表

1. 空闲链表，目标是记录空闲块
2. 隐式空闲链表
3. 显式空闲链表

北京大学
PEKING UNIVERSITY

PART 02

Implicit free list

- *Header & footer*
- *Splitting & coalescing*

隐式空闲链表

1. Size和a/f利用同一个word (为什么可以？)
2. 地址是double word对齐，对于32位系统，一个word四个byte 对于64位系统，一个word八个byte，双字对齐，16个byte，一个地址存一个byte，所以对齐之后，size是16的倍数，也即10000的倍数，末四位天然是0；我可以把这四位用起来。
3. 其实用最后一位就可以了，1表示allocated，0表示free.

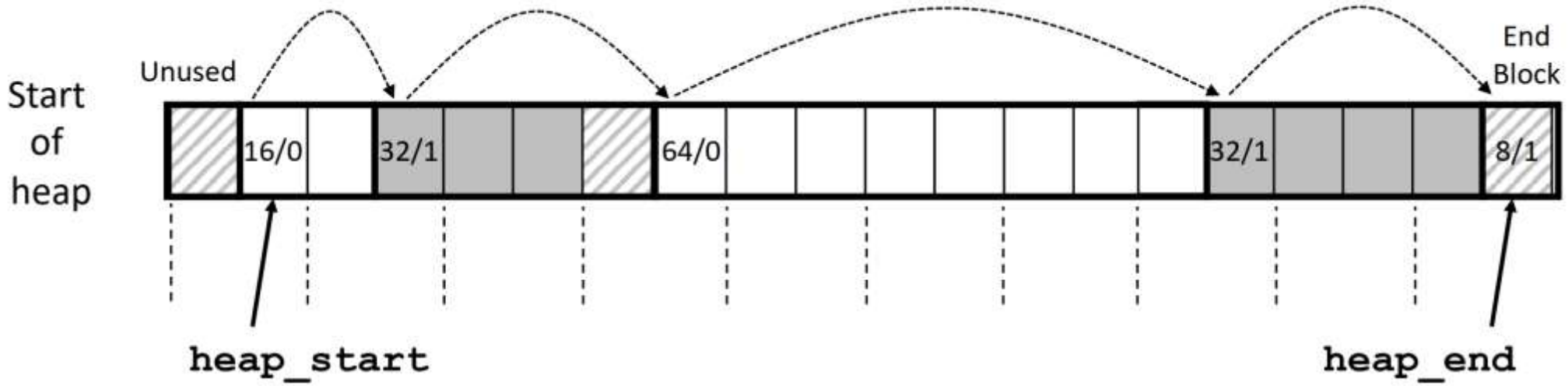
a = 1: Allocated block
a = 0: Free block

Size: total block size

Payload: application data (allocated blocks only)

隐式空闲链表

1. 之所以叫做隐式空闲链表，是因为空闲块是通过头部中的大小字段隐含地连接着的。分配器可以通过遍历堆中所有的块，从而间接地遍历整个空闲块的集合。
2. 模拟一下，到头了怎么办？
3. 设置了已分配位（a）而size=0的终止头部(terminating header)

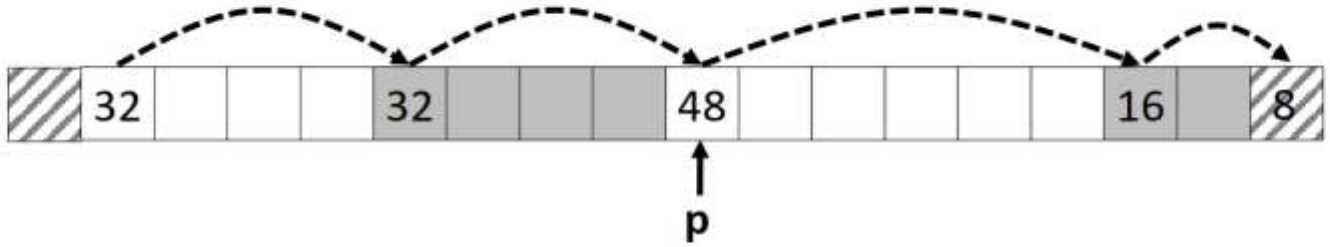


从这一个到下一个?

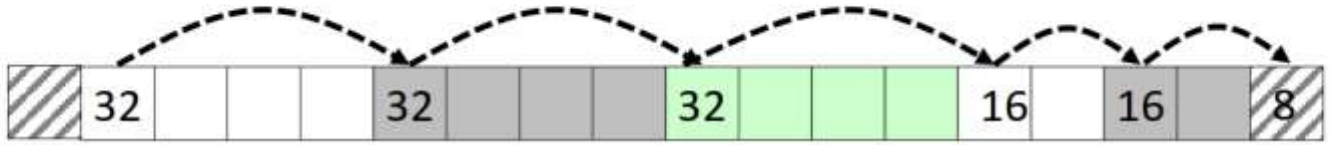
```
const size_t dsize = 2*sizeof(word_t);
static word_t *header_to_footer(block_t *block)
{
    size_t asize = get_size(block);
    return (word_t *) (block->payload + asize - dsize);
}
```

分配（分割）与合并

- Since allocated space might be smaller than free space, we might want to split the block

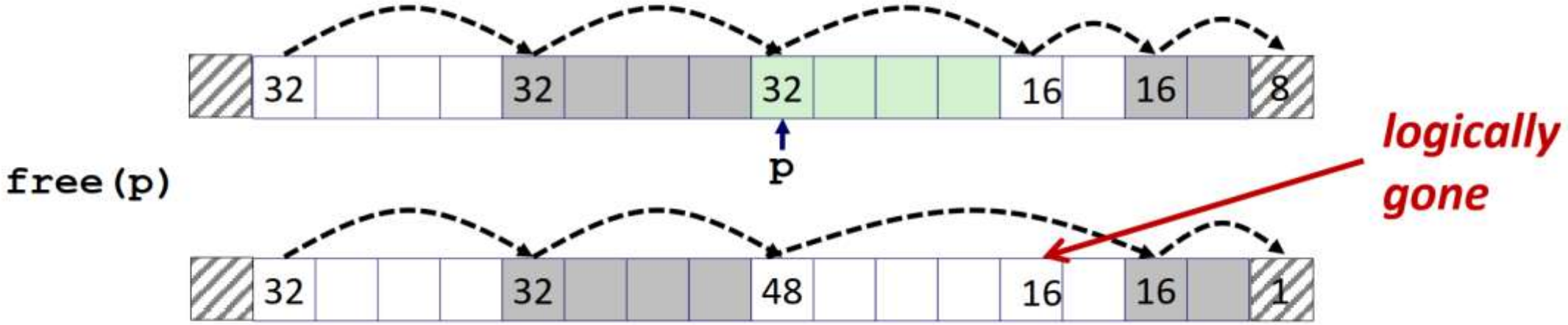



```
split_block(p, 32)
```



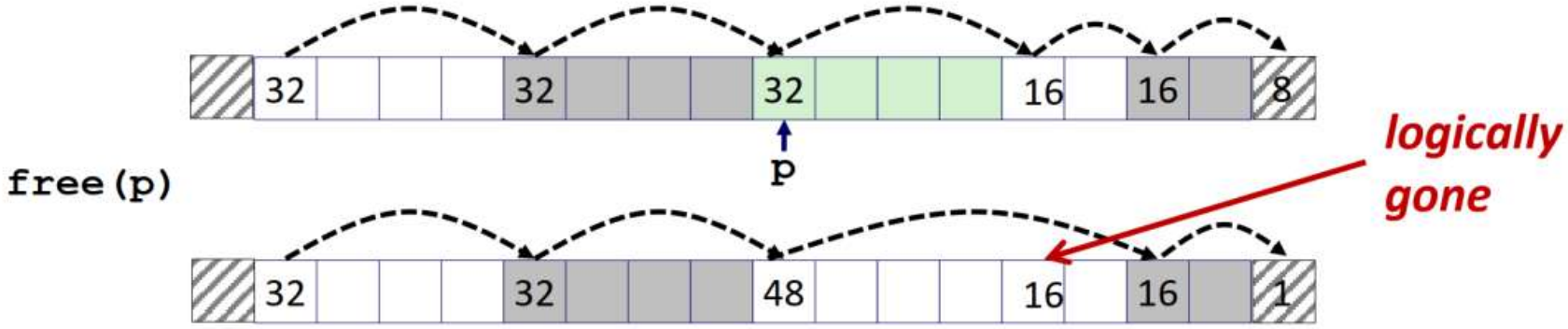
释放要考虑合并

- Coalescing with next block



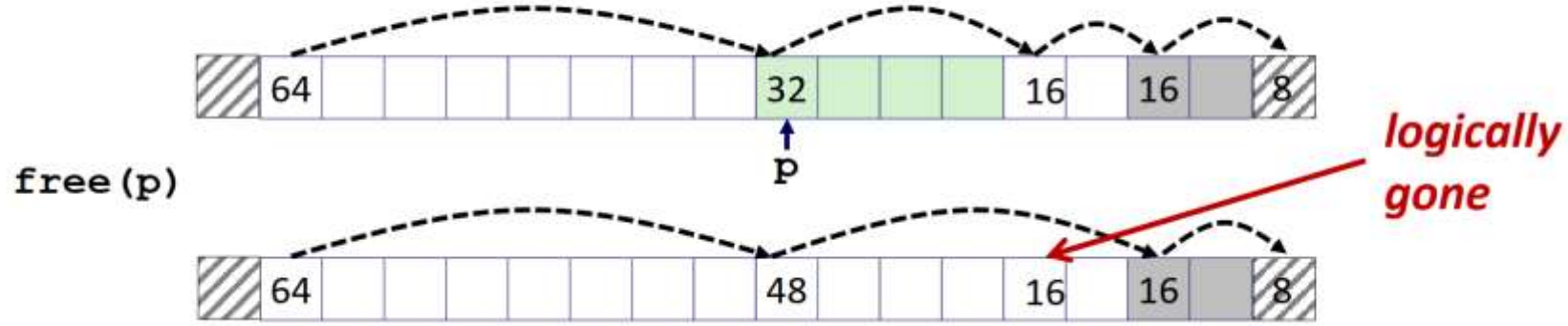
释放要考虑合并

- Coalescing with next block

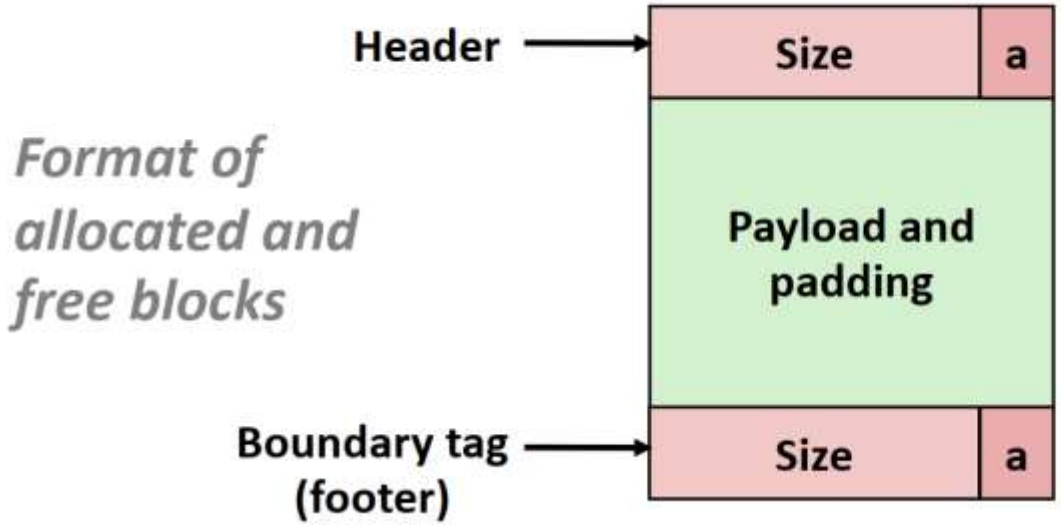


但是

- Coalescing with next block



怎么改?

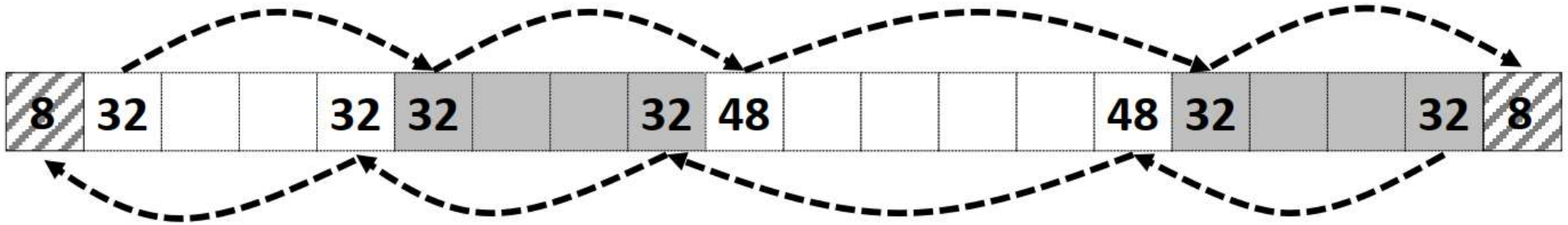


Format of allocated and free blocks

a = 1: Allocated block
a = 0: Free block

Size: Total block size

Payload: Application data (allocated blocks only)



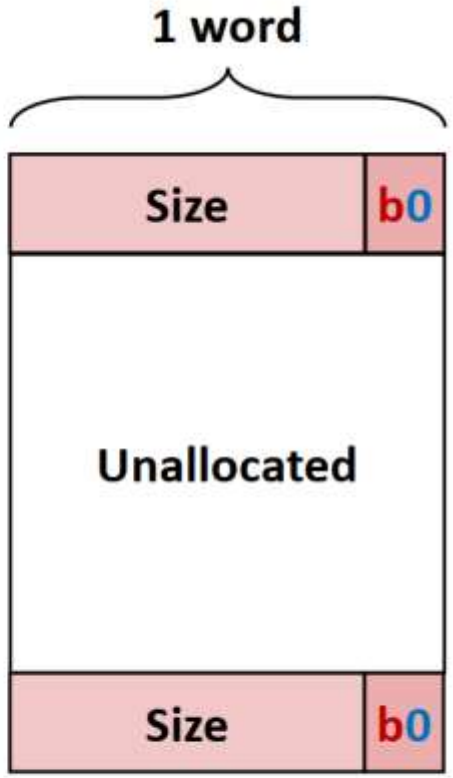
更进一步

Allocated Block

a = 1: Allocated block
a = 0: Free block
b = 1: Previous block is allocated
b = 0: Previous block is free

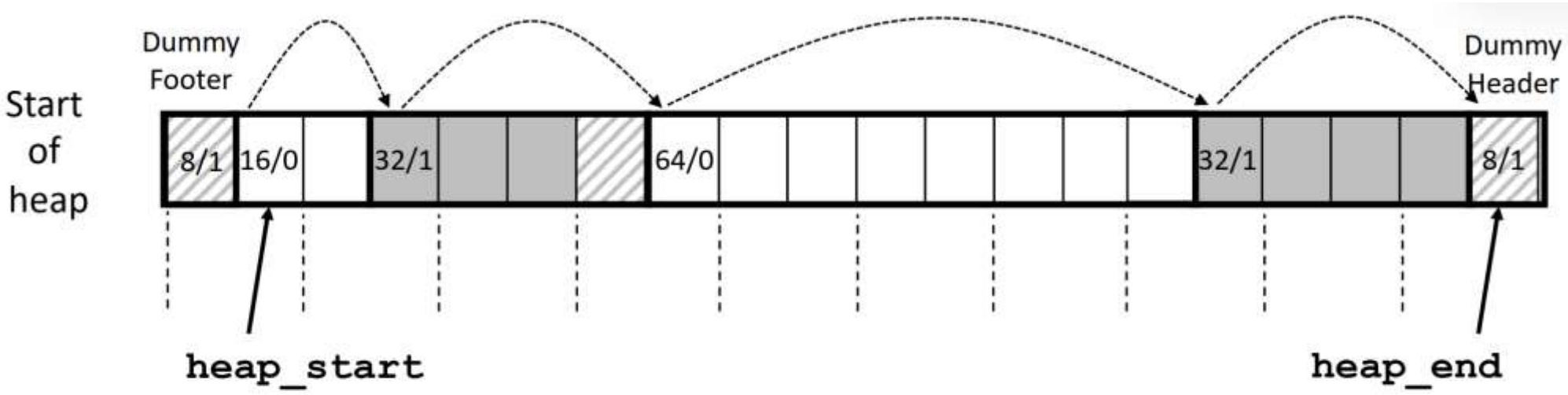
Size: block size

Payload: application data



Free Block

隐式空闲链表



- **Dummy footer before first header**
  - Marked as allocated
  - Prevents accidental coalescing when freeing first block
- **Dummy header after last footer**
  - Prevents accidental coalescing when freeing final block

北京大学
PEKING UNIVERSITY

PART 03

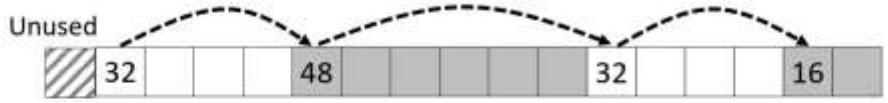
Explicit free list

- 和隐式空闲链表的区别？
- 数据结构？
- 优势？

显式空闲链表和隐式空闲链表的区别

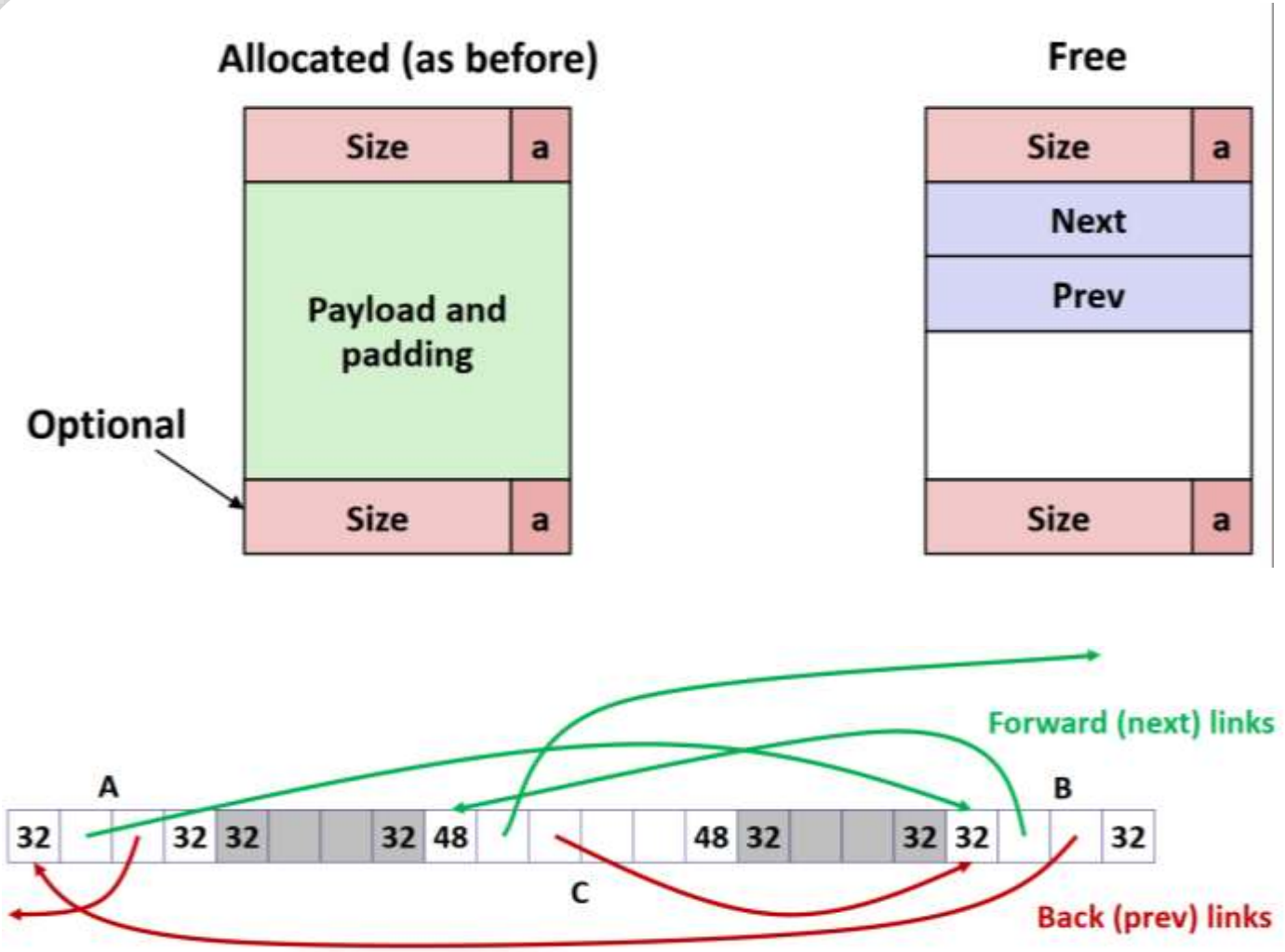
1. 使用双向链表而不是隐式空闲链表，使首次适配的分配时间从块总数的线性时间减少到了空闲块数量的线性时间。释放一个块的时间可以是线性的，也可能是常数。
2. 怎么维护呢？链表，有祖先和后继，而且有链表头Root
3. LIFO(Last In First Out), FIFO(First In First Out)

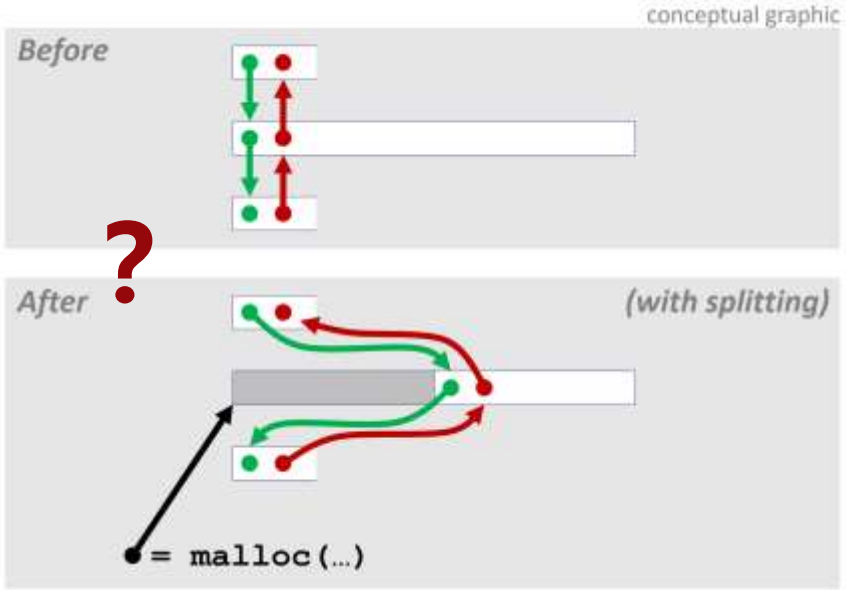
- **Method 1: *Implicit list* using length—links all blocks**



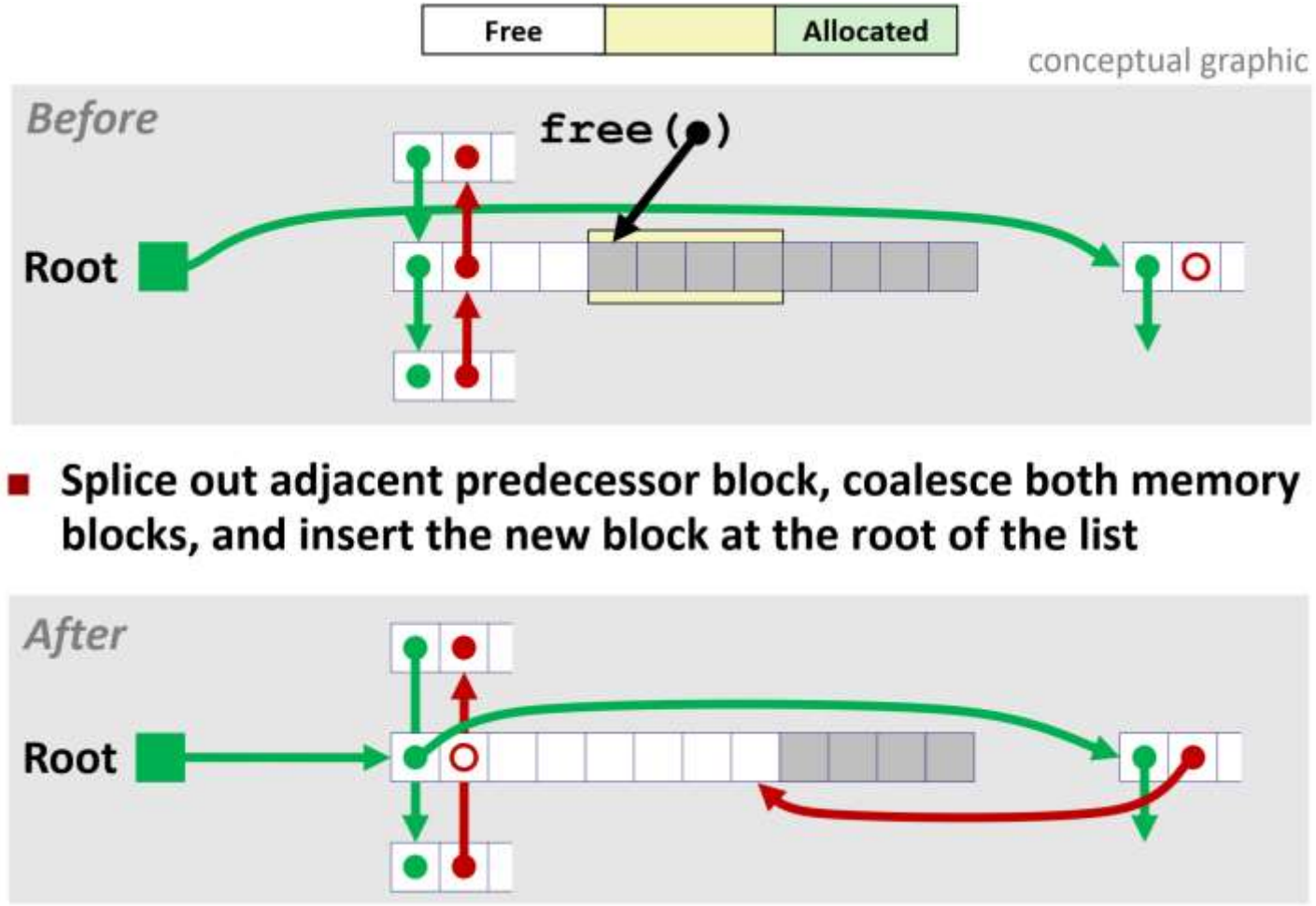
- **Method 2: *Explicit list* among the free blocks using pointers**

塞入prev和next



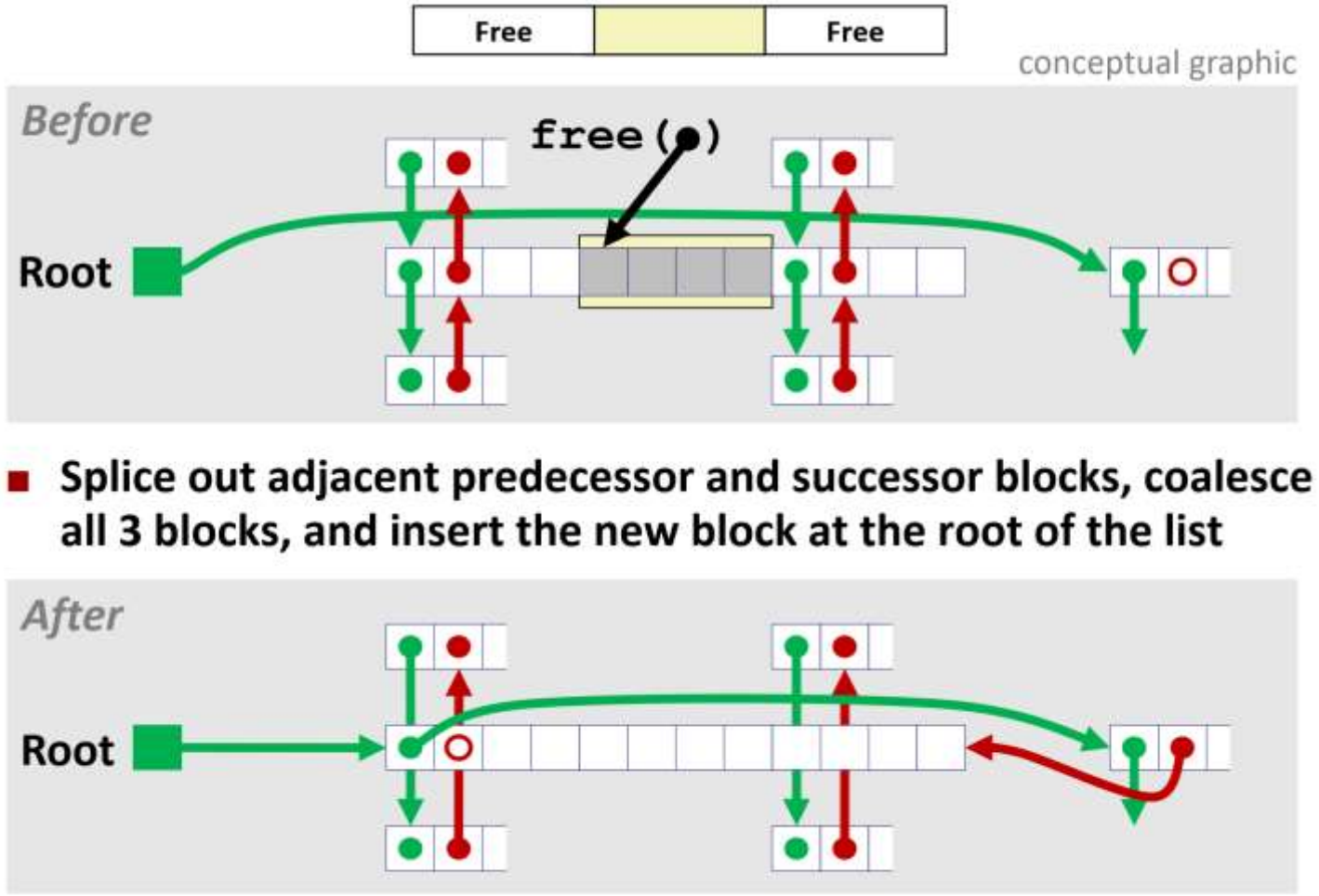


按这种画法理解一下allocate



1. Allocate is linear time in number of **free** blocks instead of **all** blocks
2. Much faster when most of the memory is full
一般而言，堆越满，隐式空闲链表要遍历的长度越长，越慢

按这种画法理解一下free



1. 注意前后两个一起合并的两个block，next和prev的变化
2. 以及那2个block的prev和next block的prev和next的变化更新

北京大学 PEKING UNIVERSITY

PART 04

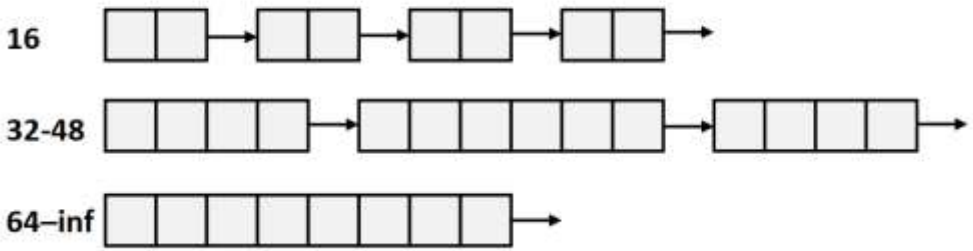
Segregated free list

- 大小类 & 空闲链表数组
- 和显式空闲链表相比，会面临什么新问题？
- 分割、合并、放置

Seglist allocator

分配器维护着一个空闲链表，每个大小类一个空闲链表，按照大小的升序排列。

Each *size class* of blocks has its own free list



To allocate a block of size n:

Search appropriate free list for block of size $m > n$ (i.e., first fit)

If an appropriate block is found, Split block and **place fragment on appropriate list**

If no block is found, try next larger class (为什么?); Repeat until block is found

If no block is found:

Request additional heap memory from OS (using **sbrk()**) (这个堆不够了)

Allocate block of *n* bytes from this new memory

Place remainder as a single free block in appropriate size class.

Seglist allocator(续)

To free a block:
Coalesce and place on appropriate list

Advantages of seglist allocators vs. non-seglist allocators (both with first-fit)

1. Higher throughput
    - log time for power-of-two size classes vs. linear time
2. Better memory utilization
    - First-fit search of segregated free list approximates a best-fit search of entire heap.
    - Extreme case: Giving each block its own size class is equivalent to best-fit.

北京大学
PEKING UNIVERSITY

PART 05

程序设计中和内存有关的坑

坑一览

```
int val;
scanf("%d", val);
```

```
int *matvec(int **A, int *x)
{
      int *y = malloc(N*sizeof(int));
      int i, j;
      for (i=0; i<N; i++)
            for (j=0; j<N; j++)
                  y[i] += A[i][j]*x[j];

      return y;
}
```

```
char *p;
p = malloc(strlen(s));
strcpy(p,s);
```

```
int **p;
p = malloc(N*sizeof(int));
for (i=0; i<N; i++)
{
   p[i] = malloc(M*sizeof(int));
}
```

```
char **p;
p = malloc(N*sizeof(int *));
for (i=0; i<=N; i++) {
p[i] = malloc(M*sizeof(int));
}
```

```
int *search(int *p, int val)
{
      while (p && *p != val)
            p += sizeof(int);
      return p;
}
```

```
int *foo ()
{
   int val;
   return &val;
}
```

坑一览（续）

```
void foo()
{
  int *x = malloc(N*sizeof(int));
  return;
}
```

```
x = malloc(N*sizeof(int));
<manipulate x>
free(x);
...
y = malloc(M*sizeof(int));
for (i=0; i<M; i++) y[i] = x[i]++;
```

北京大学
PEKING UNIVERSITY

PART 06

Implicit memory allocator: garbage collection

垃圾收集器

A garbage collector views memory as a directed *reachability graph* of the form shown in Figure 9.49. The nodes of the graph are partitioned into a set of *root nodes* and a set of *heap nodes*. Each heap node corresponds to an allocated block in the heap. A directed edge $p \rightarrow q$ means that some location in block p points to some location in block q. Root nodes correspond to locations not in the heap that contain pointers into the heap. These locations can be registers, variables on the stack, or global variables in the read/write data area of virtual memory.

We say that a node p is *reachable* if there exists a directed path from any root node to p. At any point in time, the unreachable nodes correspond to garbage that can never be used again by the application. The role of a garbage collector is to maintain some representation of the reachability graph and periodically reclaim the unreachable nodes by freeing them and returning them to the free list.

垃圾收集器

Figure 9.49
A garbage collector's view of memory as a directed graph.

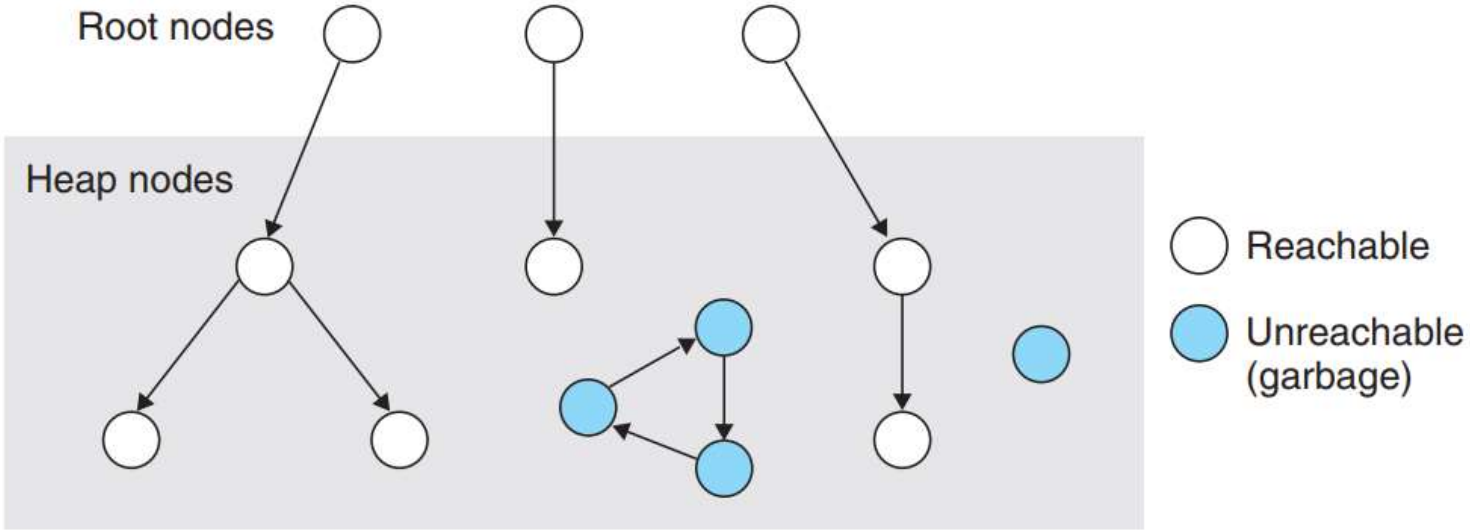
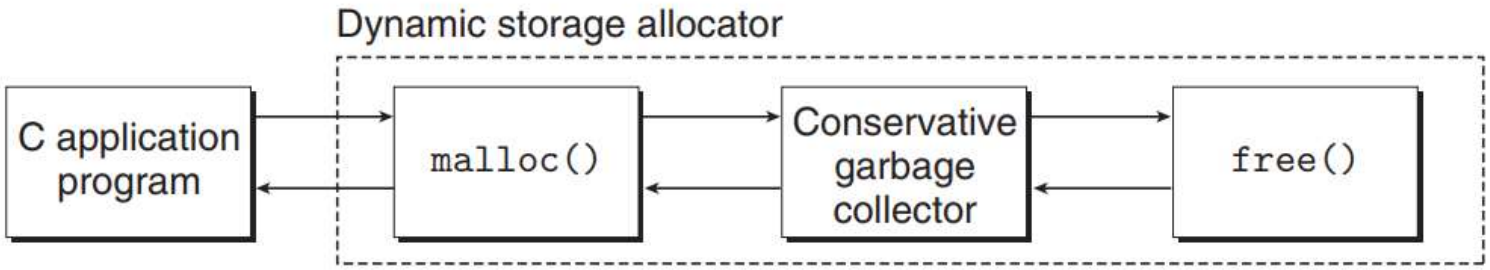


Figure 9.50
Integrating a conservative garbage collector and a C `malloc` package.



Thanks

Dec 10，2025